SUMMARY: PANDAS-II

SESSION OVERVIEW:

In this session, the students will be able to:

- Understand attributes of a DataFrame
- Understand the operations on rows and columns in DataFrames
- Understand joining, merging, and concatenation of DataFrames.

DataSets used for this session:

1. Products
2. Purchases
3. Customers

KEY POINTS AND EXAMPLES:

Undertsand the attributes of DataFrames

Like Series, we can access certain properties called attributes of a DataFrame by using that property with the DataFrame name.

1. **index:** Returns an Index object containing the row labels of the DataFrame.

```
# Accessing the index attribute
print("Index of the Customers DataFrame:")
Customers.index
```

The output will be:

```
Index of the Customers DataFrame:
RangeIndex(start=0, stop=1000, step=1)
```

The output indicates that the DataFrame has 1000 rows, with index labels from 0 to 999.

2. **shape:** Returns a tuple representing the dimensions of the DataFrame (number of rows, number of columns).

```
print("Shape of the Customers DataFrame:", Customers.shape)
```

The output will be:

```
Shape of the Customers DataFrame: (1000, 11)
```

- dataFrame.shape[0] gives you the number of rows.
- dataFrame.shape[1] gives you the number of columns

```
Customers.shape[0].
```

The output will be:

```
1000
```

3. **columns:** Returns an Index object containing the column labels of the DataFrame.

```
print("Column labels of the Customers DataFrame:",
Customers.columns)
```

The output will be:

```
Column labels of the Customers DataFrame: Index(['id', 'first_name', 'last_name', 'email', 'gender', 'street_num',
       'street_name', 'street_suffix', 'city', 'state', 'postcode'],
      dtype='object')
```

4. **dtypes:** Returns a Series with the data types of each column in the DataFrame.

```
print("Data types of each column in the Customers DataFrame:")
Customers.dtypes
```

The output will be:

```
Data types of each column in the Customers DataFrame:
id                 int64
first_name        object
last_name         object
email             object
gender            object
street_num       float64
street_name       object
street_suffix     object
city              object
state             object
postcode         float64
dtype: object
```

```
Purchase.dtypes
```

The output will be:

```
Unnamed: 0       int64
id               int64
purch_date      object
customer_num     int64
product_num      int64
amount           int64
paid            object
dtype: object
```

```
Products.dtypes
```

```
Unnamed: 0     int64
id             int64
product       object
cost          object
company       object
dtype: object
```

We can determine the data type of individual columns within a DataFrame

```
# Print the data type of the 'state' column in the Customers
DataFrame
print(Customers['state'].dtypes)
```

The output will be:

```
object
```

5. **values:** Returns a 2D NumPy array representing the data in the DataFrame.

```
print("Values in the Customers DataFrame:")
print(Customers.values)
```

The output will be:

```
Values in the Customers DataFrame:
[[1 'Romain' 'Southcott' ... 'San Diego' 'California' 92127.0]
 [2 'Cosimo' 'Molyneaux' ... 'El Paso' 'Texas' nan]
 [3 'Bambi' 'Westrip' ... 'San Antonio' 'Texas' 78220.0]
 ...
 [998 'Tobit' 'Birt' ... 'Waterbury' 'Connecticut' 6721.0]
 [999 'Issiah' 'Standbrooke' ... 'Savannah' 'Georgia' 31405.0]
 [1000 'Elmore' 'Malpas' ... 'Atlanta' 'Georgia' 30386.0]]
```

6. **size:** Returns the number of elements in the DataFrame (number of rows * number of columns).

```
print("Number of elements in the Customers DataFrame:",
Customers.size)
```

The output will be:

```
Number of elements in the Customers DataFrame: 11000
```

7. **empty:** Returns True if the DataFrame is empty (contains no rows or columns), otherwise False.

```
print("Is the Customers DataFrame empty?", Customers.empty)
```

The output will be:

```
Is the Customers DataFrame empty? False
```

8. **ndim:** Returns the number of dimensions of the DataFrame. For DataFrame, it's always 2.

```
print("Number of dimensions of the Customers DataFrame:", Customers.ndim)
```

The output will be:

```
Number of dimensions of the Customers DataFrame: 2
```

9. hasnans
10. empty

Attributes for Pandas DataFrame but not for Series:

1. .columns
2. .dtypes
3. .ftypes
4. .memory_usage
5. .style
6. .loc
7. .iloc
8. .at (same as .loc)
9. .iat (same as .iloc)

Below are the methods that can be applied on dataframe

1. **info():** Provides a concise summary of the DataFrame including data types, non-null values, and memory usage.

```
print("Summary information about the Customers DataFrame:")
Customers.info()
```

The output will be:

```
Summary information about the Customers DataFrame:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 11 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   id             1000 non-null   int64
 1   first_name     1000 non-null   object
 2   last_name      1000 non-null   object
 3   email          878 non-null    object
 4   gender         957 non-null    object
 5   street_num     738 non-null    float64
 6   street_name    963 non-null    object
 7   street_suffix  963 non-null    object
 8   city           921 non-null    object
 9   state          920 non-null    object
 10  postcode       843 non-null    float64
dtypes: float64(2), int64(1), object(8)
memory usage: 86.1+ KB
```

2. **head(n)**: Returns the first n rows of the DataFrame. If the value for n is not passed, then by default n takes 5 and the first five members are displayed.

```
print("First 10 rows of the Customers DataFrame:")
Customers.head(10)
```

The output will be:

First 10 rows of the Customers DataFrame:

	id	first_name	last_name	email	gender	street_num	street_name	street_suffix	city	state	postcode
0	1	Romain	Southcott	rsouthcott0@clickbank.net	Male	1.0	Trailsway	Road	San Diego	California	92127.0
1	2	Cosimo	Molyneaux	cmolyneaux1@wiley.com	Male	NaN	NaN	NaN	El Paso	Texas	NaN
2	3	Bambi	Westrip	bwestrip2@symantec.com	Female	4057.0	Arkansas	Circle	San Antonio	Texas	78220.0
3	4	Roarke	Pankettman	rpankettman3@wiley.com	Male	74.0	Debs	Point	Memphis	Tennessee	38150.0
4	5	Mikaela	Althorpe	malthorpe4@51.la	NaN	NaN	2nd	Drive	NaN	NaN	83732.0
5	6	Magdalena	Cullip	mcullip5@tiny.cc	Female	9190.0	Packers	Drive	Baltimore	Maryland	21229.0
6	7	Marietta	Heball	mheball6@blog.com	Female	12.0	Mallory	Center	Carol Stream	Illinois	60351.0
7	8	Tine	McSperrin	tmcsperrin7@statcounter.com	NaN	530.0	Erie	Plaza	Kansas City	Missouri	64193.0
8	9	Enrichetta	de Villier	edevillier8@ox.ac.uk	Female	745.0	Annamark	Street	Fairbanks	Alaska	99709.0
9	10	Sari	Poulden	spoulden9@xing.com	Female	2.0	Pond	Hill	New Orleans	Louisiana	NaN

3. **tail(n):** Returns the last n rows of the DataFrame. If the value for n is not passed, then by default n takes 5 and the last five members are displayed.

```
print("Last 5 rows of the Customers DataFrame:")
Customers.tail()
```

The output will be:

```
Last 5 rows of the Customers DataFrame:
```

	id	first_name	last_name	email	gender	street_num	street_name	street_suffix	city	state	postcode
995	996	Merrili	Alman	malmanrn@cornell.edu	Female	0.0	Thompson	Way	Reading	Pennsylvania	19610.0
996	997	Winonah	Heckle	whecklero@fc2.com	Female	701.0	Rowland	Hill	Indianapolis	Indiana	46278.0
997	998	Tobit	Birt	NaN	Male	2.0	Basil	Road	Waterbury	Connecticut	6721.0
998	999	Issiah	Standbrooke	istandbrookerq@yellowpages.com	Male	NaN	Kenwood	Drive	Savannah	Georgia	31405.0
999	1000	Elmore	Malpas	NaN	Male	205.0	Farwell	Park	Atlanta	Georgia	30386.0

4. **describe():** Generates descriptive statistics summary of the DataFrame.

```
print("Descriptive statistics summary of the Customers DataFrame:")
Customers.describe()
```

The output will be:

```
Descriptive statistics summary of the Customers DataFrame:
```

	id	street_num	postcode
count	1000.000000	738.000000	843.000000
mean	500.500000	10536.439024	52669.548043
std	288.819436	23050.537603	28140.041026
min	1.000000	0.000000	214.000000
25%	250.750000	21.000000	29279.500000
50%	500.500000	445.500000	48232.000000
75%	750.250000	6918.500000	78337.500000
max	1000.000000	99918.000000	99812.000000

5. **transpose():** Swaps the rows and columns of the DataFrame.

```
print("Transposed Customers DataFrame:")
Customers.transpose()
```

The output will be:

Transposed Customers DataFrame:

	0	1	2	3	4	5	
id	1	2	3	4	5	6	
first_name	Romain	Cosimo	Bambi	Roarke	Mikaela	Magdalena	Mar
last_name	Southcott	Molyneaux	Westrip	Pankettman	Althorpe	Cullip	He
email	rsouthcott0@clickbank.net	cmolyneaux1@wiley.com	bwestrip2@symantec.com	rpankettman3@wiley.com	malthorpe4@51.la	mcullip5@tiny.cc	mheball6@blog.
gender	Male	Male	Female	Male	NaN	Female	Fer
street_num	1.0	NaN	4057.0	74.0	NaN	9190.0	
street_name	Trailsway	NaN	Arkansas	Debs	2nd	Packers	Ma
street_suffix	Road	NaN	Circle	Point	Drive	Drive	Ce
city	San Diego	El Paso	San Antonio	Memphis	NaN	Baltimore	Carol Str
state	California	Texas	Texas	Tennessee	NaN	Maryland	Illi
postcode	92127.0	NaN	78220.0	38150.0	83732.0	21229.0	603

11 rows × 1000 columns

Understand the operations on rows and columns in DataFrames

We can perform some basic operations on rows and columns of a DataFrame like selection, deletion, addition, and renaming.

Columns can be referred to as attributes or features.
Rows can be referred to as records.

A) **Adding a New Column to a DataFrame:** Adding new columns to a DataFrame in pandas is a common operation. It allows you to enrich your data by adding new attributes or features based on existing data or external data.

```
# Adding a column with a scalar value
Customers['Country'] = 'USA'
print("Customers DataFrame with Country column:")
Customers
```

In this example:

- Customers['Country'] = 'USA' adds a new column named 'Country' to the DataFrame.
- The value 'USA' is assigned to every row in the 'Country' column.
- The modified DataFrame Customers will now include the new 'Country' column with 'USA' populated for each row.

The output will be:

Customers DataFrame with Country column:

	id	first_name	last_name	email	gender	street_num	street_name	street_suffix	city	state	postcode	Country
0	1	Romain	Southcott	rsouthcott0@clickbank.net	Male	1.0	Trailsway	Road	San Diego	California	92127.0	USA
1	2	Cosimo	Molyneaux	cmolyneaux1@wiley.com	Male	NaN	NaN	NaN	El Paso	Texas	NaN	USA
2	3	Bambi	Westrip	bwestrip2@symantec.com	Female	4057.0	Arkansas	Circle	San Antonio	Texas	78220.0	USA
3	4	Roarke	Pankettman	rpankettman3@wiley.com	Male	74.0	Debs	Point	Memphis	Tennessee	38150.0	USA
4	5	Mikaela	Althorpe	malthorpe4@51.la	NaN	NaN	2nd	Drive	NaN	NaN	83732.0	USA
...	...	...	...	...	...	...	...	...	...	...	...	...
995	996	Merrili	Alman	malmanrn@cornell.edu	Female	0.0	Thompson	Way	Reading	Pennsylvania	19610.0	USA
996	997	Winonah	Heckle	whecklero@fc2.com	Female	701.0	Rowland	Hill	Indianapolis	Indiana	46278.0	USA
997	998	Tobit	Birt	NaN	Male	2.0	Basil	Road	Waterbury	Connecticut	6721.0	USA
998	999	Issiah	Standbrooke	istandbrookerq@yellowpages.com	Male	NaN	Kenwood	Drive	Savannah	Georgia	31405.0	USA
999	1000	Elmore	Malpas	NaN	Male	205.0	Farwell	Park	Atlanta	Georgia	30386.0	USA

1000 rows × 12 columns

```
# Adding a column from a list
membership_status = ['Gold', 'Silver', 'Gold', 'Bronze', 'Silver']
Customers['Membership_Status'] = membership_status
print("Customers DataFrame with Membership_Status column:")
Customers
```

The output will be:

```
ValueError: Length of values (5) does not match length of index
(1000)
```

This means that when you try to assign values from membership_status to the new column 'Membership_Status', pandas expects the length of membership_status to be the same as the number of rows in your DataFrame.

```
# Adding a column based on another column
Customers['Full_Name'] = Customers['first_name'] + ' ' +
Customers['last_name']
print("Customers DataFrame with Full_Name column:")
Customers
```

Customers['first_name'] + ' ' + Customers['last_name']: This concatenates the values in the 'first_name' and 'last_name' columns for each row in the DataFrame, separated by a space ' '

The output will be:

Customers DataFrame with Full_Name column:

rst_name	last_name	email	gender	street_num	street_name	street_suffix	city	state	postcode	Country	Full_Name
Romain	Southcott	rsouthcott0@clickbank.net	Male	1.0	Trailsway	Road	San Diego	California	92127.0	USA	Romain Southcott
Cosimo	Molyneaux	cmolyneaux1@wiley.com	Male	NaN	NaN	NaN	El Paso	Texas	NaN	USA	Cosimo Molyneaux
Bambi	Westrip	bwestrip2@symantec.com	Female	4057.0	Arkansas	Circle	San Antonio	Texas	78220.0	USA	Bambi Westrip
Roarke	Pankettman	rpankettman3@wiley.com	Male	74.0	Debs	Point	Memphis	Tennessee	38150.0	USA	Roarke Pankettman
Mikaela	Althorpe	malthorpe4@51.la	NaN	NaN	2nd	Drive	NaN	NaN	83732.0	USA	Mikaela Althorpe
...	...	...	...	...	...	...	...	...	...	...	...

B) **Adding a New Row to a DataFrame:** Adding a new row to a DataFrame in pandas involves using .loc by specifying the index label directly

```
Customers.loc['1000'] = [1001,'Romain', 'Westrip',
'bwestrip52@symantec.com', 'Male', 90.0, None,None, None,
'Georgia', 83732.0, 'USA', 'Romain Westrip']
Customers.tail()
```

In this example:

- Customers.loc['1000']: This specifies the index label ('1000') where the new row will be added.
- [1001, 'Romain', 'Westrip', 'bwestrip52@symantec.com', 'Male', 90.0, None, None, None, 'Georgia', 83732.0, 'USA', 'Romain Westrip']: This list contains the values for each column in the new row.
- After executing this line, a new row will be added to the Customers DataFrame with index label '1000' and the corresponding values in each column.

Note: Make sure that the length and order of values in the list match the columns in the DataFrame, and the index label ('1000' in this case) doesn't already exist to avoid overwriting existing data.

The output will be:

	id	first_name	last_name	email	gender	street_num	street_name	street_suffix	city	state	postcode	Country
996	997	Winonah	Heckle	whecklero@fc2.com	Female	701.0	Rowland	Hill	Indianapolis	Indiana	46278.0	USA
997	998	Tobit	Birt	NaN	Male	2.0	Basil	Road	Waterbury	Connecticut	6721.0	USA
998	999	Issiah	Standbrooke	istandbrookerq@yellowpages.com	Male	NaN	Kenwood	Drive	Savannah	Georgia	31405.0	USA
999	1000	Elmore	Malpas	NaN	Male	205.0	Farwell	Park	Atlanta	Georgia	30386.0	USA
1000	1001	Romain	Westrip	bwestrip52@symantec.com	Male	90.0	None	None	None	Georgia	83732.0	USA

DataFrame.loc[] method can also be used to change the data values of a row to a particular

value.

```
Customers.loc['1000'] = None
Customers.tail()
```

If you execute Customers.loc['1000'] = None and then check the last few rows of the Customers DataFrame using Customers.tail(), you will likely see that the row at index label '1000' will be filled with None values across all columns.Please note that if index label '1000' already exists in the DataFrame, this operation will overwrite the existing row with None values.

C) **Deleting Rows or Columns from a DataFrame:** Deleting Rows or Columns from a DataFrame in pandas can be achieved using the drop() method.

To delete one or more rows from a DataFrame, you can use the drop() method along with the index labels or integer positions of the rows you want to remove.

```
Customers.drop(index='1000', inplace=True)
```

In this example, the row with index label '1000' will be removed from the Customers DataFrame.

When you want to delete multiple rows from a DataFrame using the drop() method in pandas, you need to pass a list of index labels to the index parameter.

```
Customers.drop(index=Customers.index[3])
```

Customers.drop(index=Customers.index[3]) returns a new DataFrame with the row at index 3 removed.

Note: inplace=True is not specified in Customers.drop(index=Customers.index[3]), the operation does not modify the original DataFrame. Instead, it returns a new DataFrame with the specified row (in this case, the row at index 3) removed.

The output will be:

	id	first_name	last_name	email	gender	street_num	street_name	street_suffix	city	state	postcode	Count
0	1.0	Romain	Southcott	rsouthcott0@clickbank.net	Male	1.0	Trailsway	Road	San Diego	California	92127.0	US
1	2.0	Cosimo	Molyneaux	cmolyneaux1@wiley.com	Male	NaN	NaN	NaN	El Paso	Texas	NaN	US
2	3.0	Bambi	Westrip	bwestrip2@symantec.com	Female	4057.0	Arkansas	Circle	San Antonio	Texas	78220.0	US
4	5.0	Mikaela	Althorpe	malthorpe4@51.la	NaN	NaN	2nd	Drive	NaN	NaN	83732.0	US

To delete one or more columns from a DataFrame, you can use the drop() method with the columns parameter.

```
columns_to_drop = ['Country', 'Full_Name']
Customers.drop(columns=columns_to_drop, inplace=True)
Customers
```

In this example:

- columns_to_drop is a list containing the names of the columns you want to drop.
- Customers.drop(columns=columns_to_drop, inplace=True) removes the columns from the Customers DataFrame. Again, inplace=True modifies the original DataFrame Customers.

The output will be:

	id	first_name	last_name	email	gender	street_num	street_name	street_suffix	city	state	postcode
0	1	Romain	Southcott	rsouthcott0@clickbank.net	Male	1.0	Trailsway	Road	San Diego	California	92127.0
1	2	Cosimo	Molyneaux	cmolyneaux1@wiley.com	Male	NaN	NaN	NaN	El Paso	Texas	NaN
2	3	Bambi	Westrip	bwestrip2@symantec.com	Female	4057.0	Arkansas	Circle	San Antonio	Texas	78220.0
3	4	Roarke	Pankettman	rpankettman3@wiley.com	Male	74.0	Debs	Point	Memphis	Tennessee	38150.0
4	5	Mikaela	Althorpe	malthorpe4@51.la	NaN	NaN	2nd	Drive	NaN	NaN	83732.0
...	...	...	...	...	...	...	...	...	...	...	...
995	996	Merrili	Alman	malmanrn@cornell.edu	Female	0.0	Thompson	Way	Reading	Pennsylvania	19610.0
996	997	Winonah	Heckle	whecklero@fc2.com	Female	701.0	Rowland	Hill	Indianapolis	Indiana	46278.0
997	998	Tobit	Birt	NaN	Male	2.0	Basil	Road	Waterbury	Connecticut	6721.0
998	999	Issiah	Standbrooke	istandbrookerq@yellowpages.com	Male	NaN	Kenwood	Drive	Savannah	Georgia	31405.0
999	1000	Elmore	Malpas	NaN	Male	205.0	Farwell	Park	Atlanta	Georgia	30386.0

1000 rows × 11 columns

```
Purchase.drop(columns='Unnamed: 0', inplace=True)
Purchase
```

The output will be:

	id	purch_date	customer_num	product_num	amount	paid
0	1	2019-01-03 00:00:00	823	27	12	$568.92
1	2	2019-01-03 00:00:00	606	28	14	$395.36
2	3	2019-01-03 00:00:00	955	9	17	$510.17
3	4	2019-01-03 00:00:00	577	19	3	$68.49
4	5	2019-01-03 00:00:00	429	8	18	$759.42
...	...	...	...	...	...	...
5995	5996	06/20/2019	893	33	5	$411.10
5996	5997	06/20/2019	566	23	11	$178.97
5997	5998	06/20/2019	114	19	9	$205.47
5998	5999	06/20/2019	404	11	20	$429.40
5999	6000	06/20/2019	88	57	4	$274.52

6000 rows × 6 columns

```
Products.drop(columns='Unnamed: 0', inplace=True)
Products
```

The output will be:

	id	product	cost	company
0	1	Liners - Baking Cups	$6.36	Skipfire
1	2	Nori Sea Weed - Gold Label	$85.74	Dynazzy
2	3	Bar Bran Honey Nut	$65.40	Ntag
3	4	Soup - Campbells Beef Stew	$68.16	Photojam
4	5	Wine - Shiraz Wolf Blass Premium	$87.39	Eare
5	6	Wine - White, Riesling, Semi - Dry	$99.22	Livepath
6	7	Brandy - Bar	$13.83	Oloo
7	8	Onions - White	$42.19	Oozz

D) **Renaming Row Labels of a DataFrame:** We can change the labels of rows and columns in a DataFrame using the DataFrame.rename() method.

```
# Renaming the first row to 'FirstCustomer'
Customers.rename(index={0: 'FirstCustomer'})
```

rename(index={0: 'FirstCustomer'}) specifies that the row with index label 0 should be renamed to 'FirstCustomer'.

Note: inplace=True is not specified in Customers.rename(index={0: 'FirstCustomer'})

The output will be:

	id	first_name	last_name	email	gender	street_num	street_name	street_suffix	city	state	postcod
FirstCustomer	1	Romain	Southcott	rsouthcott0@clickbank.net	Male	1.0	Trailsway	Road	San Diego	California	92127.
1	2	Cosimo	Molyneaux	cmolyneaux1@wiley.com	Male	NaN	NaN	NaN	El Paso	Texas	Nal
2	3	Bambi	Westrip	bwestrip2@symantec.com	Female	4057.0	Arkansas	Circle	San Antonio	Texas	78220.
3	4	Roarke	Pankettman	rpankettman3@wiley.com	Male	74.0	Debs	Point	Memphis	Tennessee	38150.
4	5	Mikaela	Althorpe	malthorpe4@51.la	NaN	NaN	2nd	Drive	NaN	NaN	83732.

when renaming multiple rows in a DataFrame using Pandas, you typically pass a dictionary to the rename() method where keys are the current index labels and values are the new index labels you want to assign.

```
# Create a dictionary mapping old index labels to new index labels
rename_dict = {
    0: 'FirstCustomer',
    1: 'SecondCustomer',
    2: 'ThirdCustomer'
    # Add more mappings as needed
}

# Rename the rows using the dictionary
Customers.rename(index=rename_dict)
```

The output will be:

	id	first_name	last_name	email	gender	street_num	street_name	street_suffix	city	state	postc
FirstCustomer	1	Romain	Southcott	rsouthcott0@clickbank.net	Male	1.0	Trailsway	Road	San Diego	California	921
SecondCustomer	2	Cosimo	Molyneaux	cmolyneaux1@wiley.com	Male	NaN	NaN	NaN	El Paso	Texas	
ThirdCustomer	3	Bambi	Westrip	bwestrip2@symantec.com	Female	4057.0	Arkansas	Circle	San Antonio	Texas	782
3	4	Roarke	Pankettman	rpankettman3@wiley.com	Male	74.0	Debs	Point	Memphis	Tennessee	381
4	5	Mikaela	Althorpe	malthorpe4@51.la	NaN	NaN	2nd	Drive	NaN	NaN	837
...	...	...	...	...	...	...	...	...	...	...	

E) **Renaming Column Labels of a DataFrame:** Renaming column labels in a DataFrame can be done using the rename() method in Pandas.

To rename columns, we can use the rename() method with a dictionary where keys are the current column names and values are the new names

```
Customers.rename(columns={
    'street_num': 'street_number',
    'street_name': 'street_address'
})
```

Note: Ensure that all old column names specified in the dictionary exist in your DataFrame. If any column name in the dictionary does not exist, Pandas will raise a KeyError if the parameter 'errors' is set to the value 'raise' (default 'ignore').

The output will be:

	id	first_name	last_name	email	gender	street_number	street_address	street_suffix	city	state	postcode
0	1.0	Romain	Southcott	rsouthcott0@clickbank.net	Male	1.0	Trailsway	Road	San Diego	California	92127.0
1	2.0	Cosimo	Molyneaux	cmolyneaux1@wiley.com	Male	NaN	NaN	NaN	El Paso	Texas	NaN
2	3.0	Bambi	Westrip	bwestrip2@symantec.com	Female	4057.0	Arkansas	Circle	San Antonio	Texas	78220.0
3	4.0	Roarke	Pankettman	rpankettman3@wiley.com	Male	74.0	Debs	Point	Memphis	Tennessee	38150.0
4	5.0	Mikaela	Althorpe	malthorpe4@51.la	NaN	NaN	2nd	Drive	NaN	NaN	83732.0
...	...	...	...	...	...	...	...	...	...	...	...
995	996.0	Merrili	Alman	malmanrn@cornell.edu	Female	0.0	Thompson	Way	Reading	Pennsylvania	19610.0
996	997.0	Winonah	Heckle	whecklero@fc2.com	Female	701.0	Rowland	Hill	Indianapolis	Indiana	46278.0
997	998.0	Tobit	Birt	NaN	Male	2.0	Basil	Road	Waterbury	Connecticut	6721.0
998	999.0	Issiah	Standbrooke	istandbrookerq@yellowpages.com	Male	NaN	Kenwood	Drive	Savannah	Georgia	31405.0
999	1000.0	Elmore	Malpas	NaN	Male	205.0	Farwell	Park	Atlanta	Georgia	30386.0

1000 rows × 11 columns

F) **Finding Unique Values:** The unique method in pandas is used to find the unique values present in a specific column of a DataFrame. This is particularly useful for identifying all distinct values in a dataset.

```
# Finding unique values in the 'gender' column
unique_genders = Customers['gender'].unique()
print("Unique values in the 'gender' column:", unique_genders)
```

The output will be:

```
Unique values in the 'gender' column: ['Male' 'Female' nan]
```

In this example, the output shows that the 'gender' column contains the unique values 'Male', 'Female', and nan (Not a Number, which represents missing values).

Additionally, to get the count of unique values in a column, you can use the nunique() method

```
# Counting the number of unique values in the 'gender' column
count_unique_genders = Customers['gender'].nunique()
print("Number of unique values in the 'gender' column:",
count_unique_genders)
```

The count of unique values in the 'gender' column being 2 (count_unique_genders = Customers['gender'].nunique() returning 2) indicates the number of distinct non-null values present in the 'gender' column.

When you use unique() on the 'gender' column, it lists all unique values including any missing values (nan).

However, nunique() specifically counts the number of unique non-null values in the column.

nan is not counted towards the nunique() count because it represents missing data, not a distinct non-null value.

Therefore, the count of 2 indicates that there are two unique non-null values ('Male' and 'Female') in the 'gender' column of the dataset, excluding any missing values (nan).

G) **Type Conversion of Columns:** Type conversion of columns is typically achieved using the .astype() method. This method allows you to explicitly convert the data type of a column in a DataFrame to another data type.

Syntax:

```
df['column_name'] = df['column_name'].astype(new_dtype)
```

- df['column_name']: Selects the specific column in the DataFrame.
- .astype(new_dtype): Converts the data type of the selected column to new_dtype.

Common Data Types:

- Numeric Types: int, float, complex
- DateTime Types: datetime64, timedelta
- Categorical Types: category
- String Types: str, object

pd.to_numeric() or pd.to_datetime() Functions: These functions are specific utility functions in Pandas used to convert specific columns to numeric or datetime types, respectively.

As observed earlier, the data types of the purch_date and paid columns in the Purchase DataFrame need to be changed.

purch_date: This column should be converted to datetime format using pd.to_datetime().
paid: Assuming this column should represent a numeric value (like amount paid), it should be converted to a numeric type (e.g., float64 or int64).

So, the changes needed are:

- Convert purch_date to datetime format.

- Convert paid to a numeric type.

```
Purchase['purch_date'] = pd.to_datetime(Purchase['purch_date'])
```

The pd.to_datetime() function converts the purch_date column to datetime format and assigns it back to the same column Purchase['purch_date']. This operation modifies the original DataFrame Purchase directly.

```
# Convert 'paid' column to numeric
Purchase['paid'] = Purchase['paid'].astype(float)
```

The output will be:

```
ValueError: could not convert string to float: '$568.92'
```

The error ValueError: could not convert string to float: '$568.92' indicates that there are non-numeric characters (like the dollar sign $) present in paid column, which prevent it from being directly converted to a float data type using .astype(float).

To convert such columns where values are formatted as currency (or other non-numeric formats), you need to remove any non-numeric characters before converting to a numeric type.

Here's how you can handle it:

- Remove Non-Numeric Characters: Use string manipulation methods to clean up the column before conversion.
- Convert to Numeric: After cleaning, convert the column to the desired numeric type.

```
# Remove non-numeric characters and convert to numeric
Purchase['paid'] = Purchase['paid'].str.replace('[\$,]', '',
regex=True).astype(float)

# Print the updated data types to verify
print(Purchase.dtypes)
```

- str.replace('[\$,]', '', regex=True): This removes the dollar signs ($) and commas (,) from the paid column using a regular expression.
- \ : This is used to escape the $ symbol, which is a special character in regular expressions.
- astype(float): Converts the cleaned paid column to float data type.
- **regex** that controls whether the pattern is interpreted as a regular expression or as a

literal string. Default value is ‘True’

The output will be:

```
id                          int64
purch_date         datetime64[ns]
customer_num                int64
product_num                 int64
amount                      int64
paid                      float64
dtype: object
```

After converting the purch_date column from a string to a datetime data type and the paid column from a string containing dollar values to a float data type, the Purchase DataFrame looks like this:

	id	purch_date	customer_num	product_num	amount	paid
0	1	2019-01-03	823	27	12	568.92
1	2	2019-01-03	606	28	14	395.36
2	3	2019-01-03	955	9	17	510.17
3	4	2019-01-03	577	19	3	68.49
4	5	2019-01-03	429	8	18	759.42
...	...	...	...	...	...	...
5995	5996	2019-06-20	893	33	5	411.10
5996	5997	2019-06-20	566	23	11	178.97
5997	5998	2019-06-20	114	19	9	205.47
5998	5999	2019-06-20	404	11	20	429.40
5999	6000	2019-06-20	88	57	4	274.52

6000 rows × 6 columns

Note:

- Ensure that after cleaning, all values in the paid column are numeric and can be safely converted to float.
- Handling non-numeric characters is crucial to avoid ValueError when converting to numeric types.

In the Products DataFrame, the cost column needs to be changed to a numeric data type, likely float or int.

```
# Remove '$' sign and convert 'cost' column to numeric type
```

```
Products['cost'] = Products['cost'].str.replace('[\$,]', '',
regex=True).astype(float)

# Display the updated data types
print(Products.dtypes)
```

The output will be:

```
id            int64
product      object
cost        float64
company      object
dtype: object
```

After converting the cost column from strings containing dollar values to a numeric data type (float), the Products DataFrame looks like this:

	id	product	cost	company
0	1	Liners - Baking Cups	6.36	Skipfire
1	2	Nori Sea Weed - Gold Label	85.74	Dynazzy
2	3	Bar Bran Honey Nut	65.40	Ntag
3	4	Soup - Campbells Beef Stew	68.16	Photojam
4	5	Wine - Shiraz Wolf Blass Premium	87.39	Eare
5	6	Wine - White, Riesling, Semi - Dry	99.22	Livepath
6	7	Brandy - Bar	13.83	Oloo
7	8	Onions - White	42.19	Oozz
8	9	Lettuce - Baby Salad Greens	30.01	Meevee
9	10	Sambuca - Ramazzotti	88.99	Livepath
10	11	Coffee - Decafenated	21.47	Meezzy
11	12	Lamb Leg - Bone - In Nz	42.56	Photofeed

Note: The astype() function in pandas does not work directly on null values (NaN), as it expects a valid value that can be cast to the specified data type.

```
Customers['postcode'] = Customers['postcode'].astype(int)
```

The output will be:

```
IntCastingNaNError: Cannot convert non-finite values (NA or inf)
```

```
to integer
```

The error message IntCastingNaNError: Cannot convert non-finite values (NA or inf) to integer occurs because the .astype(int) method in pandas cannot directly convert non-finite values (like NaN or infinities) to integer type.

Understand joining, merging, and concatenation of DataFrames

To manipulate and combine data across multiple DataFrames in Pandas, there are primarily three methods: joining, merging, and concatenation. Each method serves a distinct purpose depending on how you want to combine your data:

1. **Concatenation:** Concatenation is used to combine DataFrames along a particular axis (either rows or columns). It is useful when you have DataFrames with the same columns or indexes and want to stack them together. Concatenation in Pandas is analogous to SQL UNION.

When to use:

- You want to combine datasets along a particular axis (rows or columns).
- Suitable for stacking datasets with the same or similar structure.

Syntax:

```
pd.concat([df1, df2, ...], axis=0)  # Concatenate along rows (default)
pd.concat([df1, df2, ...], axis=1)  # Concatenate along columns
```

```
# three DataFrames: Customers, Purchases, Products
# Concatenate row-wise (stack vertically)
combined_df = pd.concat([Customers, Purchase, Products], axis=0)
combined_df
```

Ensure that the DataFrames have the same number of columns along the concatenation axis (axis=0 in this case). If they don't, Pandas will fill missing values (NaN) for the columns that are not present in each DataFrame.

The output will be:

	id	first_name	last_name	email	gender	street_num	street_name	street_suffix	city	state	postcode	purch_date	customer_r
0	1	Romain	Southcott	rsouthcott0@clickbank.net	Male	1.0	Trailsway	Road	San Diego	California	92127.0	NaT	I
1	2	Cosimo	Molyneaux	cmolyneaux1@wiley.com	Male	NaN	NaN	NaN	El Paso	Texas	NaN	NaT	I
2	3	Bambi	Westrip	bwestrip2@symantec.com	Female	4057.0	Arkansas	Circle	San Antonio	Texas	78220.0	NaT	I
3	4	Roarke	Pankettman	rpankettman3@wiley.com	Male	74.0	Debs	Point	Memphis	Tennessee	38150.0	NaT	I
4	5	Mikaela	Althorpe	malthorpe4@51.la	NaN	NaN	2nd	Drive	NaN	NaN	83732.0	NaT	I
...	...	...	...	...	...	...	...	...	...	...	...	...	
55	56	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaT	I
56	57	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaT	I
57	58	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaT	I
58	59	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaT	I
59	60	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaT	I

7060 rows × 19 columns

When concatenating DataFrames along axis=0 (row-wise), Pandas aligns the DataFrames based on their columns. If two DataFrames have the same column names, their rows are stacked one on top of the other. If DataFrames have different column names, Pandas will create new columns to accommodate the additional data.

```
# Customers, Purchases, Products
# Concatenate column-wise (stack horizontally)
combined_df = pd.concat([Customers, Purchase, Products], axis=1)

# Print or inspect the combined DataFrame
combined_df
```

Ensure that the indices are aligned properly across all DataFrames to avoid misalignment issues. If the indices are not aligned, Pandas will fill missing values (NaN) for the rows that do not have corresponding indices in other DataFrames.

The output willl be:

	id	first_name	last_name	email	gender	street_num	street_name	street_suffix	city	state	...	id	purch_date	custom
0	1.0	Romain	Southcott	rsouthcott0@clickbank.net	Male	1.0	Trailsway	Road	San Diego	California	...	1	2019-01-03	
1	2.0	Cosimo	Molyneaux	cmolyneaux1@wiley.com	Male	NaN	NaN	NaN	El Paso	Texas	...	2	2019-01-03	
2	3.0	Bambi	Westrip	bwestrip2@symantec.com	Female	4057.0	Arkansas	Circle	San Antonio	Texas	...	3	2019-01-03	
3	4.0	Roarke	Pankettman	rpankettman3@wiley.com	Male	74.0	Debs	Point	Memphis	Tennessee	...	4	2019-01-03	
4	5.0	Mikaela	Althorpe	malthorpe4@51.la	NaN	NaN	2nd	Drive	NaN	NaN	...	5	2019-01-03	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	
5995	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	5996	2019-06-20	
5996	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	5997	2019-06-20	
5997	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	5998	2019-06-20	
5998	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	5999	2019-06-20	
5999	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	6000	2019-06-20	

6000 rows × 21 columns

When concatenating DataFrames along axis=1 (column-wise), Pandas aligns the DataFrames based on their rows. If two DataFrames have the same index, their columns are placed side by side. If DataFrames have different indices, Pandas will create new rows to accommodate the additional data, filling in NaN values for missing entries.

2. **Merging:** Merging in Pandas is analogous to SQL joins and is used to combine DataFrames based on one or more keys. It's ideal when you need to combine data that share common columns or indices.

When to use:

- You need to combine two datasets based on a common column or index.
- Similar to SQL joins (inner join, outer join, left join, right join).

Explanation of Joins

1. Inner Join:
   pd.merge(Customers, Purchase, on='id', how='inner'): This merges the Customers and Purchase DataFrames based on the common column id and returns only the rows where id values match in both DataFrames.

```
# Merge Customers and Purchase DataFrames on 'id'
merged_df = pd.merge(Customers, Purchase, on='id', how='inner')

# Display the merged DataFrame
print("Merged DataFrame:")
merged_df
```

The output will be:

```
Merged DataFrame:
```

	id	first_name	last_name	email	gender	street_num	street_name	street_suffix	city	state	postcode	purch_d
0	1	Romain	Southcott	rsouthcott0@clickbank.net	Male	1.0	Trailsway	Road	San Diego	California	92127.0	2019-01
1	2	Cosimo	Molyneaux	cmolyneaux1@wiley.com	Male	NaN	NaN	NaN	El Paso	Texas	NaN	2019-01
2	3	Bambi	Westrip	bwestrip2@symantec.com	Female	4057.0	Arkansas	Circle	San Antonio	Texas	78220.0	2019-01
3	4	Roarke	Pankettman	rpankettman3@wiley.com	Male	74.0	Debs	Point	Memphis	Tennessee	38150.0	2019-01
4	5	Mikaela	Althorpe	malthorpe4@51.la	NaN	NaN	2nd	Drive	NaN	NaN	83732.0	2019-01
...	...	...	...	...	...	...	...	...	...	...	...	
995	996	Merrili	Alman	malmanrn@cornell.edu	Female	0.0	Thompson	Way	Reading	Pennsylvania	19610.0	2019-06
996	997	Winonah	Heckle	whecklero@fc2.com	Female	701.0	Rowland	Hill	Indianapolis	Indiana	46278.0	2019-06
997	998	Tobit	Birt	NaN	Male	2.0	Basil	Road	Waterbury	Connecticut	6721.0	2019-06
998	999	Issiah	Standbrooke	istandbrookerq@yellowpages.com	Male	NaN	Kenwood	Drive	Savannah	Georgia	31405.0	2019-06
999	1000	Elmore	Malpas	NaN	Male	205.0	Farwell	Park	Atlanta	Georgia	30386.0	2019-06

1000 rows × 16 columns

2. Left Join

   pd.merge(Customers, Purchase, on='id', how='left'): This merges the Customers and Purchase DataFrames, keeping all rows from the Customers DataFrame (left DataFrame) and adding columns from Purchase where id values match. If there is no match, NaN is used.

```
# Left Join
merged_left = pd.merge(Customers, Purchase, on='id', how='left')
merged_left
```

The output will be:

	id	first_name	last_name	email	gender	street_num	street_name	street_suffix	city	state	postcode	purch_d
0	1	Romain	Southcott	rsouthcott0@clickbank.net	Male	1.0	Trailsway	Road	San Diego	California	92127.0	2019-01
1	2	Cosimo	Molyneaux	cmolyneaux1@wiley.com	Male	NaN	NaN	NaN	El Paso	Texas	NaN	2019-01
2	3	Bambi	Westrip	bwestrip2@symantec.com	Female	4057.0	Arkansas	Circle	San Antonio	Texas	78220.0	2019-01
3	4	Roarke	Pankettman	rpankettman3@wiley.com	Male	74.0	Debs	Point	Memphis	Tennessee	38150.0	2019-01
4	5	Mikaela	Althorpe	malthorpe4@51.la	NaN	NaN	2nd	Drive	NaN	NaN	83732.0	2019-01
...	...	...	...	...	...	...	...	...	...	...	...	
995	996	Merrili	Alman	malmanrn@cornell.edu	Female	0.0	Thompson	Way	Reading	Pennsylvania	19610.0	2019-06
996	997	Winonah	Heckle	whecklero@fc2.com	Female	701.0	Rowland	Hill	Indianapolis	Indiana	46278.0	2019-06
997	998	Tobit	Birt	NaN	Male	2.0	Basil	Road	Waterbury	Connecticut	6721.0	2019-06
998	999	Issiah	Standbrooke	istandbrookerq@yellowpages.com	Male	NaN	Kenwood	Drive	Savannah	Georgia	31405.0	2019-06
999	1000	Elmore	Malpas	NaN	Male	205.0	Farwell	Park	Atlanta	Georgia	30386.0	2019-06

1000 rows × 16 columns

3. Right Join

   pd.merge(Customers, Purchase, on='id', how='right'): This merges the Customers and Purchase DataFrames, keeping all rows from the Purchase DataFrame (right

DataFrame) and adding columns from Customers where id values match. If there is no match, NaN is used.

```
# Right Join
merged_right = pd.merge(Customers, Purchase, on='id', how='right')
merged_right
```

The output will be:

	id	first_name	last_name	email	gender	street_num	street_name	street_suffix	city	state	postcode	purch_date	custon
0	1	Romain	Southcott	rsouthcott0@clickbank.net	Male	1.0	Trailsway	Road	San Diego	California	92127.0	2019-01-03	
1	2	Cosimo	Molyneaux	cmolyneaux1@wiley.com	Male	NaN	NaN	NaN	El Paso	Texas	NaN	2019-01-03	
2	3	Bambi	Westrip	bwestrip2@symantec.com	Female	4057.0	Arkansas	Circle	San Antonio	Texas	78220.0	2019-01-03	
3	4	Roarke	Pankettman	rpankettman3@wiley.com	Male	74.0	Debs	Point	Memphis	Tennessee	38150.0	2019-01-03	
4	5	Mikaela	Althorpe	malthorpe4@51.la	NaN	NaN	2nd	Drive	NaN	NaN	83732.0	2019-01-03	
...	...	...	...	...	...	...	...	...	...	...	...	...	...
5995	5996	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	2019-06-20	
5996	5997	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	2019-06-20	
5997	5998	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	2019-06-20	
5998	5999	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	2019-06-20	
5999	6000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	2019-06-20	

6000 rows × 16 columns

When you merge two DataFrames, it's possible that they have columns with the same name other than the key column(s). By default, Pandas will throw an error if there are overlapping column names. The suffixes parameter allows you to handle this situation by appending specified suffixes to the overlapping column names.

Syntax:

```
pd.merge(left, right, on='key_column', how='join_type',
suffixes=('_left_suffix', '_right_suffix'))
```

Parameters:

- left: The first DataFrame to merge.
- right: The second DataFrame to merge.
- on: The key column(s) on which to merge the DataFrames.
- how: The type of join to perform ('left', 'right', 'outer', 'inner').
- suffixes: A tuple of two strings specifying the suffixes to add to overlapping column names from the left and right DataFrames.

In the DataFrames (Customers, Purchase, and Products), there are no overlapping column names apart from the key column id. Therefore, the suffixes parameter would not be necessary in this case.

Let's create a specific example that demonstrates how the suffixes parameter works when there are overlapping column names in the DataFrames being merged.

```
# Create sample DataFrames with overlapping column names
Customers = pd.DataFrame({
    'id': [1, 2, 3],
    'first_name': ['Alice', 'Bob', 'Charlie'],
    'product': ['Book', 'Pen', 'Notebook']
})

Purchase = pd.DataFrame({
    'id': [2, 3, 4],
    'product': ['Pen', 'Notebook', 'Pencil'],
    'amount': [5, 10, 15]
})

# Merge DataFrames with suffixes for overlapping columns
merged_df = pd.merge(Customers, Purchase, on='id', how='inner',
suffixes=('_cust', '_pur'))

print("Merged DataFrame with Suffixes:\n", merged_df)
```

The output will be:

```
Merged DataFrame with Suffixes:
    id first_name product_cust product_pur  amount
0   2        Bob          Pen         Pen       5
1   3    Charlie     Notebook    Notebook      10
```

In this example:

- DataFrames:
    - Customers DataFrame contains columns id, first_name, and product.
    - Purchase DataFrame contains columns id, product, and amount.
- Overlapping Column: Both DataFrames have a column named product.
- Merge Operation: We merge the DataFrames on the id column using an inner join and specify the suffixes _cust and _pur.
- suffixes=('_cust', '_pur'):
    - product column from the Customers DataFrame is renamed to product_cust.
    - product column from the Purchase DataFrame is renamed to product_pur.

3. **Joining:** Joining is a convenient method for combining columns from two potentially differently-indexed DataFrames into a single result DataFrame. It uses indexes to join DataFrames.

When to use:

- You need to combine two DataFrames based on their indexes.
- Can be thought of as a blend of merge and concat.

```
# Performing join operation on 'id' column
joined_df = Customers.set_index('id').join(Purchase)

print("Joined DataFrame:")
joined_df
```

set_index('id') is used on both DataFrames to set the id column as the index.
join() method then combines the DataFrames horizontally (column-wise) based on their indices (which are now the id column).

The output will be:

```
Joined DataFrame:
```

id	first_name	last_name	email	gender	street_num	street_name	street_suffix	city	state	postcode	id	purch_
1	Romain	Southcott	rsouthcott0@clickbank.net	Male	1.0	Trailsway	Road	San Diego	California	92127.0	2	2019-0
2	Cosimo	Molyneaux	cmolyneaux1@wiley.com	Male	NaN	NaN	NaN	El Paso	Texas	NaN	3	2019-0
3	Bambi	Westrip	bwestrip2@symantec.com	Female	4057.0	Arkansas	Circle	San Antonio	Texas	78220.0	4	2019-0
4	Roarke	Pankettman	rpankettman3@wiley.com	Male	74.0	Debs	Point	Memphis	Tennessee	38150.0	5	2019-0
5	Mikaela	Althorpe	malthorpe4@51.la	NaN	NaN	2nd	Drive	NaN	NaN	83732.0	6	2019-0
...	...	...	...	...	...	...	...	...	...	...	...	
996	Merrili	Alman	malmanrn@cornell.edu	Female	0.0	Thompson	Way	Reading	Pennsylvania	19610.0	997	2019-0
997	Winonah	Heckle	whecklero@fc2.com	Female	701.0	Rowland	Hill	Indianapolis	Indiana	46278.0	998	2019-0
998	Tobit	Birt	NaN	Male	2.0	Basil	Road	Waterbury	Connecticut	6721.0	999	2019-0
999	Issiah	Standbrooke	istandbrookerq@yellowpages.com	Male	NaN	Kenwood	Drive	Savannah	Georgia	31405.0	1000	2019-0
1000	Elmore	Malpas	NaN	Male	205.0	Farwell	Park	Atlanta	Georgia	30386.0	1001	2019-0

1000 rows × 16 columns

key differences between concatenation, joining, and merging

Features	**Concatenation**	**Merging**	**Joining**
Purpose	Stack DataFrames along an axis	Combine DataFrames based on keys	Combine DataFrames based on index
Function	concat()	merge()	join()

Axis	axis=0 (rows), axis=1 (columns)	N/A	N/A
Data Alignment	Based on index or columns	Based on specified key columns	Based on index or specified key column
Key Column	Not required	Requires common key column(s)	Requires a common key column or index
Flexibility	Limited to stacking along an axis	Join on multiple columns	Join on indices or columns
Column Names	Must be identical for axis=1	Not necessarily identical	Not necessarily identical
Usage	Combine DataFrames with similar structures	Combine DataFrames with complex key relationships	Combine DataFrames with shared index or key

Understand how to handle missing values

As we know that a DataFrame can consist of many rows (objects) where each row can have values for various columns (attributes). If a value corresponding to a column is not present, it is considered to be a missing value. A missing value is denoted by NaN.

In the real world dataset, it is common for an object to have some missing attributes. There may be several reasons for that. In some cases, data was not collected properly resulting in missing data e.g some people did not fill all the fields while taking the survey. Sometimes, some attributes are not relevant to all. For example, if a person is unemployed then salary attribute will be irrelevant and hence may not have been filled up.

Missing values create a lot of problems during data analysis and have to be handled properly. The two most common strategies for handling missing values are:

1. Drop the Row or Column with Missing Values
2. Fill or estimate the missing value

Before proceeding with any strategy to handle missing values, it's crucial to first identify where these missing values are located within dataset. By checking missing values, you get a clear picture of which columns or rows have missing data.

Checking Missing Values

Pandas provides two functions, isnull() and isna(), to check for missing values (NaN or None) in a DataFrame:

1. isnull(): This function returns a boolean DataFrame of the same shape as the original DataFrame. Each element is True if the corresponding element in the original DataFrame is NaN or None, and False otherwise.

2. isna(): Similar to isnull(), isna() also returns a boolean DataFrame with the same shape as the original DataFrame. It performs the same operation as isnull() and is essentially an alias for it.

They help in identifying missing data, which is crucial for data cleaning and preprocessing tasks in data analysis.

```
# Using isnull() to check for missing values
null_values = Customers.isnull()
print("Using isnull():")
print(null_values)

# Using isna() to check for missing values (is equivalent to
isnull())
na_values = Customers.isna()
print("\nUsing isna():")
print(na_values)
```

In this example:

- isnull() and isna() functions are used to identify missing values in the Customers DataFrame.
- Both functions will return a DataFrame where each element is True if the corresponding element in the original DataFrame is NaN or None, and False otherwise. both isnull() and isna() functions generate the same result.
- The output will show True or False values indicating the presence or absence of missing values in each cell of the DataFrame.

The output will be:

```
Using isnull():
        id  first_name  last_name  email  gender  street_num  street_name  \
0    False       False      False  False   False       False        False
1    False       False      False  False   False        True         True
2    False       False      False  False   False       False        False
3    False       False      False  False   False       False        False
4    False       False      False  False    True        True        False
..     ...         ...        ...    ...     ...         ...          ...
995  False       False      False  False   False       False        False
996  False       False      False  False   False       False        False
997  False       False      False   True   False       False        False
998  False       False      False  False   False        True        False
999  False       False      False   True   False       False        False

     street_suffix   city  state  postcode
0            False  False  False     False
1             True  False  False      True
2            False  False  False     False
3            False  False  False     False
4            False   True   True     False
..             ...    ...    ...       ...
995          False  False  False     False
996          False  False  False     False
997          False  False  False     False
998          False  False  False     False
999          False  False  False     False

[1000 rows x 11 columns]
```

```
Using isna():
        id  first_name  last_name  email  gender  street_num  street_name  \
0    False       False      False  False   False       False        False
1    False       False      False  False   False        True         True
2    False       False      False  False   False       False        False
3    False       False      False  False   False       False        False
4    False       False      False  False    True        True        False
..     ...         ...        ...    ...     ...         ...          ...
995  False       False      False  False   False       False        False
996  False       False      False  False   False       False        False
997  False       False      False   True   False       False        False
998  False       False      False  False   False        True        False
999  False       False      False   True   False       False        False

     street_suffix   city  state  postcode
0            False  False  False     False
1             True  False  False      True
2            False  False  False     False
3            False  False  False     False
4            False   True   True     False
..             ...    ...    ...       ...
995          False  False  False     False
996          False  False  False     False
997          False  False  False     False
998          False  False  False     False
999          False  False  False     False

[1000 rows x 11 columns]
```

To check for missing values in each individual attribute (column) of a DataFrame

```
print(Customers['email'].isnull())
```

The output will be a Series of boolean values, where each element corresponds to a row in the ‘email’' column.

The output will be:

```
0      False
1      False
2      False
3      False
4      False
       ...
995    False
996    False
997     True
998    False
999     True
Name: email, Length: 1000, dtype: bool
```

To check whether a column (attribute) has a missing value in the entire dataset, any() function is used. It returns True in case of missing value else returns False.

```
# Check for missing values in each column of the Customers
DataFrame
missing_in_columns = Customers.isnull().any()

# Print the result
print("Missing values in each column:")
print(missing_in_columns)
```

The output will be:

```
Missing values in each column:
id              False
first_name      False
last_name       False
email            True
gender           True
street_num       True
street_name      True
street_suffix    True
city             True
state            True
postcode         True
dtype: bool
```

The function any() can be used for a particular attribute also.

```
print(Customers['street_suffix'] .isnull().any())
```

The output will be:

```
True
```

df.isnull().any().any() returns True if there are any missing values (NaN) in the DataFrame df, and it returns False if there are no missing values.

```
# Check if any element in the boolean DataFrame
(missing_in_columns) is True
any_element_true = Customers.isnull().any().any()
any_element_true
```

The output will be:

```
True
```

To find the number of NaN (missing) values corresponding to each attribute (column) in a Pandas DataFrame, you can use the sum() function in conjunction with the isnull() function.

```
# Calculate the number of NaN values in each attribute (column) of
the Customers DataFrame
nan_counts_per_attribute = Customers.isnull().sum()

# Print the result
print("Number of NaN values per attribute:")
print(nan_counts_per_attribute)
```

- Customers.isnull(): This function generates a boolean DataFrame where each element is True if the corresponding element in the original DataFrame (Customers) is NaN, and False otherwise.
- .sum(): Applied to the boolean DataFrame returned by isnull(), this function sums up the True values along each column (axis=0 by default), because True is treated as 1 and False as 0 in numerical operations.
- nan_counts_per_attribute: This variable stores the resulting Series where each index represents an attribute (column name) from the original DataFrame (Customers), and each corresponding value represents the count of NaN values in that attribute.

The output will display the number of NaN values for each attribute (column) in the Customers DataFrame. Each attribute name will be followed by its corresponding count of NaN values.

The output will be:

```
Number of NaN values per attribute:
id                0
first_name        0
last_name         0
email           122
gender           43
street_num      262
street_name      37
street_suffix    37
city             79
state            80
postcode        157
dtype: int64
```

```
# Calculate the number of NaN values in each attribute (column) of the Purchase DataFrame
nan_counts_purchase = Purchase.isnull().sum()

# Calculate the number of NaN values in each attribute (column) of the Products DataFrame
nan_counts_products = Products.isnull().sum()

# Print the results
print("Number of NaN values per attribute in Purchase:")
print(nan_counts_purchase)

print("\nNumber of NaN values per attribute in Products:")
print(nan_counts_products)
```

The output will be:

```
Number of NaN values per attribute in Purchase:
id              0
purch_date      0
customer_num    0
product_num     0
amount          0
paid            0
dtype: int64

Number of NaN values per attribute in Products:
id         0
product    0
cost       0
company    5
dtype: int64
```

To find the number of NaN values for each row in the DataFrame:

```
nan_counts_per_row = Customers.isnull().sum(axis=1)
print("Number of NaN values per row:")
print(nan_counts_per_row)
```

.sum(axis=1): This sums the True values (missing values) along each row of the DataFrame because axis=1 specifies that the operation should be performed along rows.

The output will be:

```
Number of NaN values per row:
0      0
1      4
2      0
3      0
4      4
      ..
995    0
996    0
997    1
998    1
999    1
Length: 1000, dtype: int64
```

To find the total number of NaN values in the whole dataset

```
total_nan = Customers.isnull().sum().sum()
print(total_nan)
```

The output will be:

```
817
```

```
total_nan_Products = Products.isnull().sum().sum()
print(total_nan_Products)
```

The output will be:

```
5
```

Note: If you encounter values in a column that cannot be converted to a specific data type, you can handle these situations gracefully by replacing such values with pd.NA.

For example, some entries in paid column contain non-numeric values like ‘unknown’', which cannot be directly converted to float. Your goal is to convert the 'Paid' column to numeric (float), handling such cases gracefully by replacing non-convertible values with pd.NA.

```
# Replace 'unknown' with pd.NA in 'Paid' column
df['Paid'] = df['Paid'].replace('unknown', pd.NA)
```

Or you can use pd.to_numeric() to achieve the same result of converting the 'Paid' column to numeric (float) and replacing non-convertible values like 'unknown' with pd.NA

Convert 'Paid' column to numeric, replace 'unknown' with pd.NA

```
df['Paid'] = pd.to_numeric(df['Paid'], errors='coerce')
```

In pandas, when you use pd.to_numeric() function to convert a column to numeric, the errors='coerce' parameter specifies how to handle errors that occur during the conversion process.
Specifically, errors='coerce' instructs pandas to:

- Convert problematic values: It attempts to convert values to numeric format.
- Set non-convertible values to NaN: If a value cannot be converted to a numeric type (e.g., due to non-numeric strings like 'unknown'), it replaces those values with NaN (Not a Number).

Dropping Missing Values

When to Use: This approach is suitable when the number of missing values is relatively small compared to the total size of the dataset or when those rows or columns do not significantly impact the analysis.

Dropping data may lead to loss of information, especially if the missing values are systematic or provide meaningful insights when imputed.

Dropping Rows with Missing Values: To drop rows that contain any missing values, you can use the dropna() function with the default parameters.

```
# Creating a copy of the original DataFrame
Customers_copy = Customers.copy()

# Dropping rows with any missing values in the copy
Customers_dropped_rows = Customers_copy.dropna()
print("DataFrame after dropping rows with any missing values
(original DataFrame unaffected):")
Customers_dropped_rows
```

The output will be:

```
DataFrame after dropping rows with any missing values (original DataFrame unaffected):
```

	id	first_name	last_name	email	gender	street_num	street_name	street_suffix	city	state	postcode
0	1.0	Romain	Southcott	rsouthcott0@clickbank.net	Male	1.0	Trailsway	Road	San Diego	California	92127.0
2	3.0	Bambi	Westrip	bwestrip2@symantec.com	Female	4057.0	Arkansas	Circle	San Antonio	Texas	78220.0
3	4.0	Roarke	Pankettman	rpankettman3@wiley.com	Male	74.0	Debs	Point	Memphis	Tennessee	38150.0
5	6.0	Magdalena	Cullip	mcullip5@tiny.cc	Female	9190.0	Packers	Drive	Baltimore	Maryland	21229.0
6	7.0	Marietta	Heball	mheball6@blog.com	Female	12.0	Mallory	Center	Carol Stream	Illinois	60351.0
...	...	...	...	...	...	...	...	...	...	...	...
988	989.0	Vivianne	Abbyss	vabbyssrg@squarespace.com	Female	53592.0	Larry	Road	Santa Cruz	California	95064.0
992	993.0	Johnath	Clancy	jclancyrk@smugmug.com	Female	7.0	Washington	Crossing	Juneau	Alaska	99812.0
994	995.0	Brana	Dixon	bdixonrm@myspace.com	Female	97.0	Truax	Avenue	Maple Plain	Minnesota	55572.0
995	996.0	Merrili	Alman	malmanrn@cornell.edu	Female	0.0	Thompson	Way	Reading	Pennsylvania	19610.0
996	997.0	Winonah	Heckle	whecklero@fc2.com	Female	701.0	Rowland	Hill	Indianapolis	Indiana	46278.0

418 rows × 11 columns

Dropping Columns with Missing Values: To drop columns with missing values in Pandas DataFrame, we can use the dropna() method with axis=1.

```
# Create a copy of the original DataFrame
Customers_copy = Customers.copy()

# Drop columns with any missing values from the copy
Customers_dropped_columns = Customers_copy.dropna(axis=1)

print("DataFrame after dropping columns with any missing values:")
Customers_dropped_columns
```

The output will be:

```
DataFrame after dropping columns with any missing values:
```

	id	first_name	last_name
0	1.0	Romain	Southcott
1	2.0	Cosimo	Molyneaux
2	3.0	Bambi	Westrip
3	4.0	Roarke	Pankettman
4	5.0	Mikaela	Althorpe
...	...	...	...
995	996.0	Merrili	Alman
996	997.0	Winonah	Heckle
997	998.0	Tobit	Birt
998	999.0	Issiah	Standbrooke
999	1000.0	Elmore	Malpas

1000 rows × 3 columns

Estimating (Imputing) Missing Values

When to Use: This method is preferred when the missing values are limited and can be reasonably estimated or imputed without compromising the analysis.

Missing values can be filled by using estimations or approximations e.g a value just before (or after) the missing value, average/minimum/maximum of the values of that attribute, etc. In some cases, missing values are replaced by zeros (or ones).

The fillna(num) function can be used to replace missing value(s) by the value specified in num. For example, fillna(0) replaces missing value by 0. Similarly fillna(1) replaces missing value by 1.

One approach is to use forward fill (ffill) or backward fill (bfill), where missing values are replaced with the last known value before or the next available value after the NaNs in a series. Alternatively, mean, median, or mode imputation replaces NaNs with the average, middle, or most frequent value of the attribute, respectively, suitable for numerical and categorical data. Another method is zero or one imputation, which replaces missing values with zeros or ones, appropriate for binary or count data.

In the Customers DataFrame:
id, first_name, last_name: These columns have no missing values (NaN), so no action is needed.

email: Since email is likely unique per customer and has 122 missing values, it might not be suitable to fill them with a simple statistic. Dropping rows with missing emails or filling with a default value (like "Unknown") could be considered.

```
# Fill missing values in 'email' column with 'Unknown'
Customers['email'].fillna('Unknown', inplace=True)

# Print or check the updated DataFrame
Customers
```

fillna('Unknown'): This method replaces missing (NaN) values in the email column with the string 'Unknown'.

The output will be:

	id	first_name	last_name	email	gender	street_num	street_name	street_suffix	city	state	postcode
0	1.0	Romain	Southcott	rsouthcott0@clickbank.net	Male	1.0	Trailsway	Road	San Diego	California	92127.0
1	2.0	Cosimo	Molyneaux	cmolyneaux1@wiley.com	Male	NaN	NaN	NaN	El Paso	Texas	NaN
2	3.0	Bambi	Westrip	bwestrip2@symantec.com	Female	4057.0	Arkansas	Circle	San Antonio	Texas	78220.0
3	4.0	Roarke	Pankettman	rpankettman3@wiley.com	Male	74.0	Debs	Point	Memphis	Tennessee	38150.0
4	5.0	Mikaela	Althorpe	malthorpe4@51.la	NaN	NaN	2nd	Drive	NaN	NaN	83732.0
...	...	...	...	...	...	...	...	...	...	...	...
995	996.0	Merrili	Alman	malmanrn@cornell.edu	Female	0.0	Thompson	Way	Reading	Pennsylvania	19610.0
996	997.0	Winonah	Heckle	whecklero@fc2.com	Female	701.0	Rowland	Hill	Indianapolis	Indiana	46278.0
997	998.0	Tobit	Birt	Unknown	Male	2.0	Basil	Road	Waterbury	Connecticut	6721.0
998	999.0	Issiah	Standbrooke	istandbrookerq@yellowpages.com	Male	NaN	Kenwood	Drive	Savannah	Georgia	31405.0
999	1000.0	Elmore	Malpas	Unknown	Male	205.0	Farwell	Park	Atlanta	Georgia	30386.0

1000 rows × 11 columns

gender: With 43 missing values, filling with the most frequent category (mode) might be appropriate, assuming it's a categorical variable.

```
# Calculate the mode of the 'gender' column
mode_gender = Customers['gender'].mode()[0]
# Fill missing values in 'gender' column with the mode
Customers['gender'].fillna(mode_gender, inplace=True)

# Print or check the updated DataFrame
Customers
```

mode_gender = Customers['gender'].mode()[0]: This line calculates the mode (most frequent value) of the gender column using the mode() function. The [0] indexing is used to extract the mode value from the Pandas Series object returned by mode().

fillna(mode_gender, inplace=True): This method replaces missing (NaN) values in the gender column with mode_gender, which is the most frequent category.

Filling missing values with mean or median is common for numerical columns.
Mean: Use when you believe the data is evenly distributed without many extreme values.
Median: Use when you believe the data may have outliers or is not symmetrically distributed.

The output will be:

	id	first_name	last_name	email	gender	street_num	street_name	street_suffix	city	state	postcode
0	1	Romain	Southcott	rsouthcott0@clickbank.net	Male	1.0	Trailsway	Road	San Diego	California	92127.0
1	2	Cosimo	Molyneaux	cmolyneaux1@wiley.com	Male	NaN	NaN	NaN	El Paso	Texas	NaN
2	3	Bambi	Westrip	bwestrip2@symantec.com	Female	4057.0	Arkansas	Circle	San Antonio	Texas	78220.0
3	4	Roarke	Pankettman	rpankettman3@wiley.com	Male	74.0	Debs	Point	Memphis	Tennessee	38150.0
4	5	Mikaela	Althorpe	malthorpe4@51.la	Male	NaN	2nd	Drive	NaN	NaN	83732.0
...	...	...	...	...	...	...	...	...	...	...	...
995	996	Merrili	Alman	malmanrn@cornell.edu	Female	0.0	Thompson	Way	Reading	Pennsylvania	19610.0
996	997	Winonah	Heckle	whecklero@fc2.com	Female	701.0	Rowland	Hill	Indianapolis	Indiana	46278.0
997	998	Tobit	Birt	Unknown	Male	2.0	Basil	Road	Waterbury	Connecticut	6721.0
998	999	Issiah	Standbrooke	istandbrookerq@yellowpages.com	Male	NaN	Kenwood	Drive	Savannah	Georgia	31405.0
999	1000	Elmore	Malpas	Unknown	Male	205.0	Farwell	Park	Atlanta	Georgia	30386.0

1000 rows × 11 columns

street_num, street_name, street_suffix: These address-related columns have significant missing values (262 for street_num, 37 for street_name and street_suffix). Depending on the analysis, dropping these rows might affect data integrity, so filling with mode or using forward/backward fill might be considered.

- Forward Fill (ffill): Forward fill propagates the last valid observation forward to the next valid. In other words, it takes the previous non-NaN value and fills the NaN values with it.
- Backward Fill (bfill): Backward fill propagates the next valid observation backward to the next valid. In other words, it takes the next non-NaN value and fills the NaN values with it.

```
# Fill missing values with forward fill (ffill)
Customers['street_num'].fillna(method='ffill', inplace=True)
Customers['street_name'].fillna(method='ffill', inplace=True)
Customers['street_suffix'].fillna(method='ffill', inplace=True)

# Print or check the updated DataFrame
Customers
```

To use backward fill (bfill), replace 'ffill' with 'bfill' in the method parameter

	id	first_name	last_name	email	gender	street_num	street_name	street_suffix	city	state	postcode
0	1.0	Romain	Southcott	rsouthcott0@clickbank.net	Male	1.0	Trailsway	Road	San Diego	California	92127.0
1	2.0	Cosimo	Molyneaux	cmolyneaux1@wiley.com	Male	1.0	Trailsway	Road	El Paso	Texas	NaN
2	3.0	Bambi	Westrip	bwestrip2@symantec.com	Female	4057.0	Arkansas	Circle	San Antonio	Texas	78220.0
3	4.0	Roarke	Pankettman	rpankettman3@wiley.com	Male	74.0	Debs	Point	Memphis	Tennessee	38150.0
4	5.0	Mikaela	Althorpe	malthorpe4@51.la	Male	74.0	2nd	Drive	NaN	NaN	83732.0
...	...	...	...	...	...	...	...	...	...	...	...
995	996.0	Merrili	Alman	malmanrn@cornell.edu	Female	0.0	Thompson	Way	Reading	Pennsylvania	19610.0
996	997.0	Winonah	Heckle	whecklero@fc2.com	Female	701.0	Rowland	Hill	Indianapolis	Indiana	46278.0
997	998.0	Tobit	Birt	Unknown	Male	2.0	Basil	Road	Waterbury	Connecticut	6721.0
998	999.0	Issiah	Standbrooke	istandbrookerq@yellowpages.com	Male	2.0	Kenwood	Drive	Savannah	Georgia	31405.0
999	1000.0	Elmore	Malpas	Unknown	Male	205.0	Farwell	Park	Atlanta	Georgia	30386.0

1000 rows × 11 columns

city, state, postcode: These address-related columns also have missing values (79 for city, 80 for state, 157 for postcode). Filling with mode or using forward/backward fill could be suitable here as well.

```
# Fill missing values with backward fill (bfill)
Customers['city'].fillna(method='bfill', inplace=True)
Customers['state'].fillna(method='bfill', inplace=True)
Customers['postcode'].fillna(method='bfill', inplace=True)

# Print or check the updated DataFrame
Customers
```

The output will be:

	id	first_name	last_name	email	gender	street_num	street_name	street_suffix	city	state	postcode
0	1.0	Romain	Southcott	rsouthcott0@clickbank.net	Male	1.0	Trailsway	Road	San Diego	California	92127.0
1	2.0	Cosimo	Molyneaux	cmolyneaux1@wiley.com	Male	1.0	Trailsway	Road	El Paso	Texas	78220.0
2	3.0	Bambi	Westrip	bwestrip2@symantec.com	Female	4057.0	Arkansas	Circle	San Antonio	Texas	78220.0
3	4.0	Roarke	Pankettman	rpankettman3@wiley.com	Male	74.0	Debs	Point	Memphis	Tennessee	38150.0
4	5.0	Mikaela	Althorpe	malthorpe4@51.la	Male	74.0	2nd	Drive	Baltimore	Maryland	83732.0
...	...	...	...	...	...	...	...	...	...	...	...
995	996.0	Merrili	Alman	malmanrn@cornell.edu	Female	0.0	Thompson	Way	Reading	Pennsylvania	19610.0
996	997.0	Winonah	Heckle	whecklero@fc2.com	Female	701.0	Rowland	Hill	Indianapolis	Indiana	46278.0
997	998.0	Tobit	Birt	Unknown	Male	2.0	Basil	Road	Waterbury	Connecticut	6721.0
998	999.0	Issiah	Standbrooke	istandbrookerq@yellowpages.com	Male	2.0	Kenwood	Drive	Savannah	Georgia	31405.0
999	1000.0	Elmore	Malpas	Unknown	Male	205.0	Farwell	Park	Atlanta	Georgia	30386.0

1000 rows × 11 columns

We have filled the missing values, let's verify that there are no remaining missing values in the DataFrame.

```
Customers.isnull().sum()
```

The output will be:

```
id              0
first_name      0
last_name       0
email           0
gender          0
street_num      0
street_name     0
street_suffix   0
city            0
state           0
postcode        0
dtype: int64
```

In the Products DataFrame:

- The 'id' and 'product' columns have 0 NaN values, meaning they are complete.
- The 'cost' column also has 0 NaN values, indicating it is complete as well.
- company: This column in the Products DataFrame has 5 missing values. Strategies for filling these missing values could include using the mode (most frequent value), filling with a default value such as “Unknown” and “Other” or applying forward fill (ffill) or backward fill (bfill).

Using a default value like "Unknown" can be a straightforward approach to handle missing values in categorical columns like 'company'.

```
# Fill missing values in the 'company' column with the default
value "Unknown"
Products['company'].fillna('Unknown', inplace=True)
```

After filling missing values in the 'company' column with the default value "Unknown", the Products DataFrame looks like this

```
Products.tail(15)
```

The output will be:

	id	product	cost	company
45	46	Aromat Spice / Seasoning	61.34	Unknown
46	47	Tobasco Sauce	48.32	Browsezoom
47	48	Apple - Delicious, Golden	36.67	Skiba
48	49	Lamb - Rack	62.60	Gevee
49	50	Silicone Paper 16.5x24	63.98	Babbleopia
50	51	Ecolab - Power Fusion	75.32	Fliptune
51	52	Soup - Knorr, Country Bean	98.74	Eimbee
52	53	Chicken - Thigh, Bone In	74.06	Topiclounge
53	54	Plate - Foam, Bread And Butter	37.79	Flipstorm
54	55	Beef - Bresaola	8.55	Digitube
55	56	Peach - Fresh	20.53	Zazio
56	57	Muffin Hinge - 211n	68.63	Fanoodle
57	58	Bar Bran Honey Nut	73.05	Yakijo
58	59	Nut - Almond, Blanched, Whole	74.28	Eazzy
59	60	Table Cloth 90x90 Colour	41.22	Quinu

We have filled the missing values, let's verify that there are no remaining missing values in the DataFrame.

```
Products.isnull().sum()
```

The output will be:

```
id         0
product    0
cost       0
company    0
dtype: int64
```

The Purchase DataFrame does not have any missing values. There is no need to fill any missing values.

Filling using interpolate

Fills NaNs using linear interpolation. Used for linear filling of missing values.

df['column_name'].interpolate(method='linear', inplace=True)

Drop Duplicates

Dropping duplicates in Pandas is an operation used to remove duplicate rows from a DataFrame, ensuring each row is unique based on specified columns.

When to Use:

- Clean a dataset by removing duplicate entries.
- Ensure data integrity by keeping only one occurrence of each duplicated row.

Syntax:

```
DataFrame.drop_duplicates(inplace=False,subset=None)
```

- subset: Column label or sequence of labels to consider for identifying duplicates. By default, considers all columns.
- inplace: If True, performs operation in-place and returns None. If False, returns a new DataFrame.

To remove duplicates based on the 'first_name' column

```
# Remove duplicates based on the 'first_name' column
Customers_unique = Customers.drop_duplicates(subset='first_name')

# Display the DataFrame with duplicates removed based on
'first_name'
print("Customers DataFrame with Duplicates Removed Based on
'first_name':")
Customers_unique
```

The output will be:

```
Customers DataFrame with Duplicates Removed Based on 'first_name':
```

	id	first_name	last_name	email	gender	street_num	street_name	street_suffix	city	state	postcode
0	1	Romain	Southcott	rsouthcott0@clickbank.net	Male	1.0	Trailsway	Road	San Diego	California	92127.0
1	2	Cosimo	Molyneaux	cmolyneaux1@wiley.com	Male	1.0	Trailsway	Road	El Paso	Texas	78220.0
2	3	Bambi	Westrip	bwestrip2@symantec.com	Female	4057.0	Arkansas	Circle	San Antonio	Texas	78220.0
3	4	Roarke	Pankettman	rpankettman3@wiley.com	Male	74.0	Debs	Point	Memphis	Tennessee	38150.0
4	5	Mikaela	Althorpe	malthorpe4@51.la	Male	74.0	2nd	Drive	Baltimore	Maryland	83732.0
...	...	...	...	...	...	...	...	...	...	...	...
994	995	Brana	Dixon	bdixonrm@myspace.com	Female	97.0	Truax	Avenue	Maple Plain	Minnesota	55572.0
995	996	Merrili	Alman	malmanrn@cornell.edu	Female	0.0	Thompson	Way	Reading	Pennsylvania	19610.0
996	997	Winonah	Heckle	whecklero@fc2.com	Female	701.0	Rowland	Hill	Indianapolis	Indiana	46278.0
997	998	Tobit	Birt	Unknown	Male	2.0	Basil	Road	Waterbury	Connecticut	6721.0
999	1000	Elmore	Malpas	Unknown	Male	205.0	Farwell	Park	Atlanta	Georgia	30386.0

932 rows × 11 columns

Data Aggregations

Aggregation refers to the process of transforming a dataset by computing a single summary value (or set of values) from multiple rows or columns of data. This summary value typically represents some statistical measure or calculation applied across the dataset. Examples of

aggregation operations include calculating the sum, mean (average), median, mode, minimum, maximum.

Aggregation is fundamental in data analysis for summarizing data, identifying patterns, and deriving insights from large datasets efficiently.

```
Purchase.aggregate('max')
```

The output will be:

```
id                              6000
purch_date      2019-12-06 00:00:00
customer_num                    1000
product_num                       60
amount                            20
paid                          1990.8
dtype: object
```

To use multiple aggregate functions in a single statement

```
Products['cost'].aggregate(['max','min'])
```

The output will be:

```
max    99.54
min     6.36
Name: cost, dtype: float64
```

```
# Group by 'customer_num' and compute sum of 'amount' and 'paid'
aggregated_purchase = Purchase.groupby('purch_date').agg({
    'amount': 'sum',
    'paid': 'sum'
})

print("Aggregated Purchase DataFrame:")
print(aggregated_purchase)
```

The output will be:

```
Aggregated Purchase DataFrame:
             amount      paid
purch_date
2019-01-03      480  20468.33
2019-01-04      872  48529.13
2019-01-05      715  38643.67
2019-01-06      576  32345.29
2019-02-03      568  28183.50
...             ...       ...
2019-11-06      475  24778.05
2019-12-03      524  27691.62
2019-12-04      647  37100.59
2019-12-05      380  20845.55
2019-12-06      481  26410.08

[108 rows x 2 columns]
```

- groupby('purch_date'): Groups the Purchase DataFrame by purch_date.
- agg({'amount': 'sum', 'paid': 'sum'}): Aggregates (sums) the amount and paid columns for each purch_date.